**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ
И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

Ордена Трудового Красного Знамени федеральное государственное
бюджетное образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра математической кибернетики и информационных технологий

ОТЧЁТ ПО ЛАБОРАТОРНЫМ РАБОТАМ

по дисциплине

«Проектирование информационных систем»

Выполнил: студент группы БСТ2257
Подзоров Д.С

Проверил: преподаватель
Сергеев Андрей Евгеньевич

Москва 2026

Лабораторная работа №1

Основы языка Python. Установка и запуск Django.

Цель работы

Данная лабораторная работа посвящена знакомству с языком программирования Python — его синтаксисом, основными структурами данных и механизмом определения функций. Параллельно выполняется установка фреймворка Django и первичная настройка его административного интерфейса.

Задачи

1. Создать скрипт helloworld.py, выводящий приветственное сообщение.

2. Реализовать список студентов с функцией табличного вывода.

3. Написать функцию фильтрации студентов по среднему баллу.

4. Установить и настроить веб-фреймворк Django.

5. Исследовать административный интерфейс Django.

Ход работы

Работа начинается с создания файла helloworld.py — минимальной программы на Python, которая выводит сообщение «Hello, World!» в стандартный поток вывода. Это позволяет проверить, что интерпретатор Python корректно установлен и настроен.

Файл helloworld.py

```
# Вывод приветствия
print("Hello, world!")
```

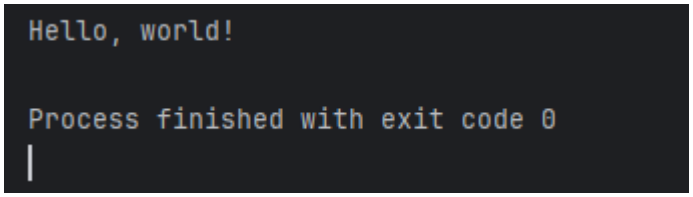
Следующим этапом был создан файл groupmates.py с массивом словарей, описывающих студентов факультета. Для каждого студента заданы поля: имя, номер группы, возраст и список оценок. В файле реализована функция

print_students для форматированного табличного вывода и функция filter_by_avg для отбора студентов с баллом выше заданного порога.

Файл groupmates.py

```
groupmates = [
    {"name": "Вячеслав", "group": "<БСТ2257>", "age": 24, "marks": [4, 3, 5, 5, 4]},
    {"name": "Дмитрий", "group": "БСТ2257", "age": 24, "marks": [5, 5, 5, 5, 4]},
    {"name": "Александр", "group": "БСТ2257", "age": 24, "marks": [3, 5, 4, 3, 5]},
    {"name": "Сергей", "group": "БСТ2257", "age": 24, "marks": [4, 5, 5, 4, 5]},
]

def print_students(students):
    print("Имя студента".ljust(15),
          "Группа".ljust(8),
          "Возраст".ljust(8),
          "Оценки".ljust(20))
    for student in students:
        print(student["name"].ljust(15),
              student["group"].ljust(8),
              str(student["age"]).ljust(8),
              str(student["marks"]).ljust(20))
    print()

def average(student):
    return sum(student["marks"]) / len(student["marks"])

def filter_students(students, min_avg):
    return [s for s in students if average(s) > min_avg]

if __name__ == "__main__":
    print_students(groupmates)
    print("Студенты со средним баллом > 4.0:")
    print_students(filter_students(groupmates, 4.0))
```

На рисунках ниже представлены результаты выполнения обоих скриптов.

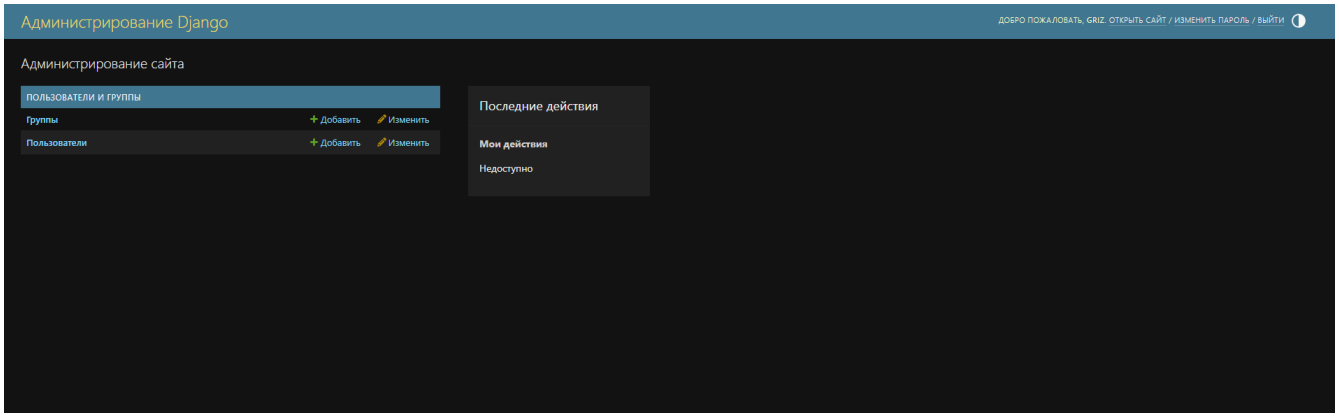


Результат выполнения helloworld.py

```
Имя студента    Группа   Возраст  Оценки
Вячеслав        <БСТ2257> 24       [4, 3, 5, 5, 4]
Дмитрий         БСТ2257  24       [5, 5, 5, 5, 4]
Александр       БСТ2257  24       [3, 5, 4, 3, 5]
Сергей          БСТ2257  24       [4, 5, 5, 4, 5]

Студенты со средним баллом > 4.0:
Имя студента    Группа   Возраст  Оценки
Вячеслав        <БСТ2257> 24       [4, 3, 5, 5, 4]
Дмитрий         БСТ2257  24       [5, 5, 5, 5, 4]
Сергей          БСТ2257  24       [4, 5, 5, 4, 5]
```

Результат выполнения groupmates.py

Также в рамках работы был установлен фреймворк Django, создан проект и выполнена первичная настройка административного интерфейса.



Административный интерфейс Django

Лабораторная работа №2

Создание первой web-страницы. HTML-шаблон. Статичные файлы.

Цель работы

Лабораторная работа направлена на освоение процесса создания web-страницы в рамках Django-проекта. Рассматриваются механизм представлений, принципы шаблонизации HTML и способы подключения статичных ресурсов — файлов стилей CSS.

Задачи

1. Создать Django-проект helloworld с приложением flatpages.

2. Написать представление для отображения главной страницы.

3. Разработать HTML-шаблон с заголовками, списками и таблицей.

4. Подключить CSS-файл для оформления страницы.

Ход работы

Файл flatpages/views.py

```
from django.shortcuts import render

def home(request):
    return render(request,'static_handler.html')
```

Шаблон index.html описывает структуру страницы: заголовки нескольких уровней, маркированный и нумерованный списки, а также HTML-таблицу. Это демонстрирует работу с основными элементами языка разметки.

Шаблон index.html

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
<title>Привет, Мир!</title>
<link rel="stylesheet" href="{% static 'css/index.css' %}">
```

```
</head>
<body>
<h1>Привет, Мир!</h1>
<h2>Это учебный сайт, с его помощью будут изучены технологии python/django, html/css.</h2>
<h3>Как видите, здесь используются заголовки различных уровней.</h3>

<img src="{% static 'img/dpb-logo.png' %}" alt="logo">

<p>Здесь есть маркированный список:</p>
<ul>
<li>Элемент 1;</li>
<li>элемент 2;</li>
<li>элемент 3;</li>
<li>последний элемент.</li>
</ul>
<p>И нумерованный список:</p>
<ol>
<li>Элемент 1;</li>
<li>элемент 2;</li>
<li>элемент 3;</li>
<li>последний элемент.</li>
</ol>
<p>И даже таблица:</p>
<table>
<thead>
<tr>
<th>Столбик 1</th>
<th>Столбик 2</th>
<th>Столбик 3</th>
<th>Столбик 4</th>
</tr>
</thead>
<tbody>
<tr><td>Строка 1 Столбец 1</td><td>Строка 1 Столбец 2</td><td>Строка 1 Столбец
3</td><td>Строка 1 Столбец 4</td></tr>
<tr><td>Строка 2 Столбец 1</td><td>Строка 2 Столбец 2</td><td>Строка 2 Столбец
3</td><td>Строка 2 Столбец 4</td></tr>
<tr><td>Строка 3 Столбец 1</td><td>Строка 3 Столбец 2</td><td>Строка 3 Столбец
3</td><td>Строка 3 Столбец 4</td></tr>
<tr><td>Строка 4 Столбец 1</td><td>Строка 4 Столбец 2</td><td>Строка 4 Столбец
3</td><td>Строка 4 Столбец 4</td></tr>
<tr><td>Строка 5 Столбец 1</td><td>Строка 5 Столбец 2</td><td>Строка 5 Столбец
3</td><td>Строка 5 Столбец 4</td></tr>
</tbody>
</table>
</body>
</html>
```

В директории static/css/ размещён файл index.css, в котором заданы стили оформления: цвет фона, семейство шрифтов, свойства таблицы и отступы элементов.

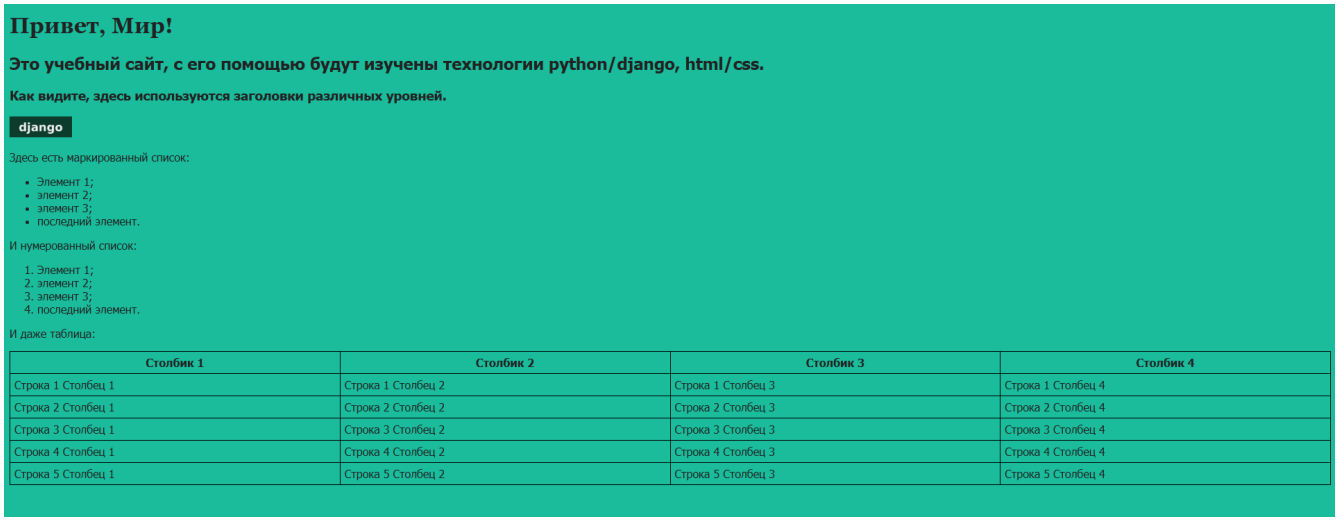
Файл static/css/index.css

```
body { background: #1abc9c; font-family: Tahoma, Arial, sans-serif; color: #222; }
h1 { font-family: Georgia, "Times New Roman", serif; }
img { height: 30px; }
table { width: 100%; border-collapse: collapse; border: 1px solid #000; }
table th, table td { border: 1px solid #000; padding: 6px; }
```

Файл urls.py

```
from django.contrib import admin
from django.urls import path
from flatpages.views import home

urlpatterns=[path('admin/', admin.site.urls), path('', home, name='home')]
```

Результат работы — страница, отображаемая в браузере:



Привет, Мир!

Это учебный сайт, с его помощью будут изучены технологии python/django, html/css.

Как видите, здесь используются заголовки различных уровней.

django

Здесь есть маркированный список:

- Элемент 1;
- элемент 2;
- элемент 3;
- последний элемент.

И нумерованный список:

1. Элемент 1;
2. элемент 2;
3. элемент 3;
4. последний элемент.

И даже таблица:

Столбик 1	Столбик 2	Столбик 3	Столбик 4
Строка 1 Столбец 1	Строка 1 Столбец 2	Строка 1 Столбец 3	Строка 1 Столбец 4
Строка 2 Столбец 1	Строка 2 Столбец 2	Строка 2 Столбец 3	Строка 2 Столбец 4
Строка 3 Столбец 1	Строка 3 Столбец 2	Строка 3 Столбец 3	Строка 3 Столбец 4
Строка 4 Столбец 1	Строка 4 Столбец 2	Строка 4 Столбец 3	Строка 4 Столбец 4
Строка 5 Столбец 1	Строка 5 Столбец 2	Строка 5 Столбец 3	Строка 5 Столбец 4

Страница «Привет, Мир!» в браузере

Лабораторная работа №3

Создание модели данных Article. Регистрация в admin. Динамический шаблон.

Цель работы

Работа посвящена созданию модели данных в Django и её интеграции с административным интерфейсом. Модели являются центральным элементом ORM-подхода, позволяя описывать структуру таблиц базы данных на уровне Python-классов. Также рассматривается динамическая генерация HTML-шаблонов с использованием контекстных переменных.

Задачи

1. Создать Django-проект blog с приложением articles.

2. Описать модель Article с полями title, author, text и created_date.

3. Настроить отображение модели в административной панели.

4. Реализовать шаблон для вывода архива всех статей.

Ход работы

Файл articles/models.py

```
from django.db import models
from django.contrib.auth.models import User

class Article(models.Model):
    title = models.CharField(max_length=200, unique=True)
    author = models.ForeignKey(User, on_delete=models.CASCADE)
    text = models.TextField()
    created_date = models.DateField(auto_now_add=True)

    def __str__(self):
        return f"{self.author.username}: {self.title}"

    def get_excerpt(self):
        return self.text[:140] + "..." if len(self.text) > 140 else self.text
```

Файл articles/admin.py

```
from django.contrib import admin
from .models import Article
```

```
@admin.register(Article)
class ArticleAdmin(admin.ModelAdmin):
    list_display=('title','author','get_excerpt','created_date')
```

Файл articles/views.py

```
from django.shortcuts import render, get_object_or_404, redirect
from django.http import Http404
from .models import Article

def archive(request):
    posts = Article.objects.all()
    return render(request, 'archive.html', {'posts': posts})
```

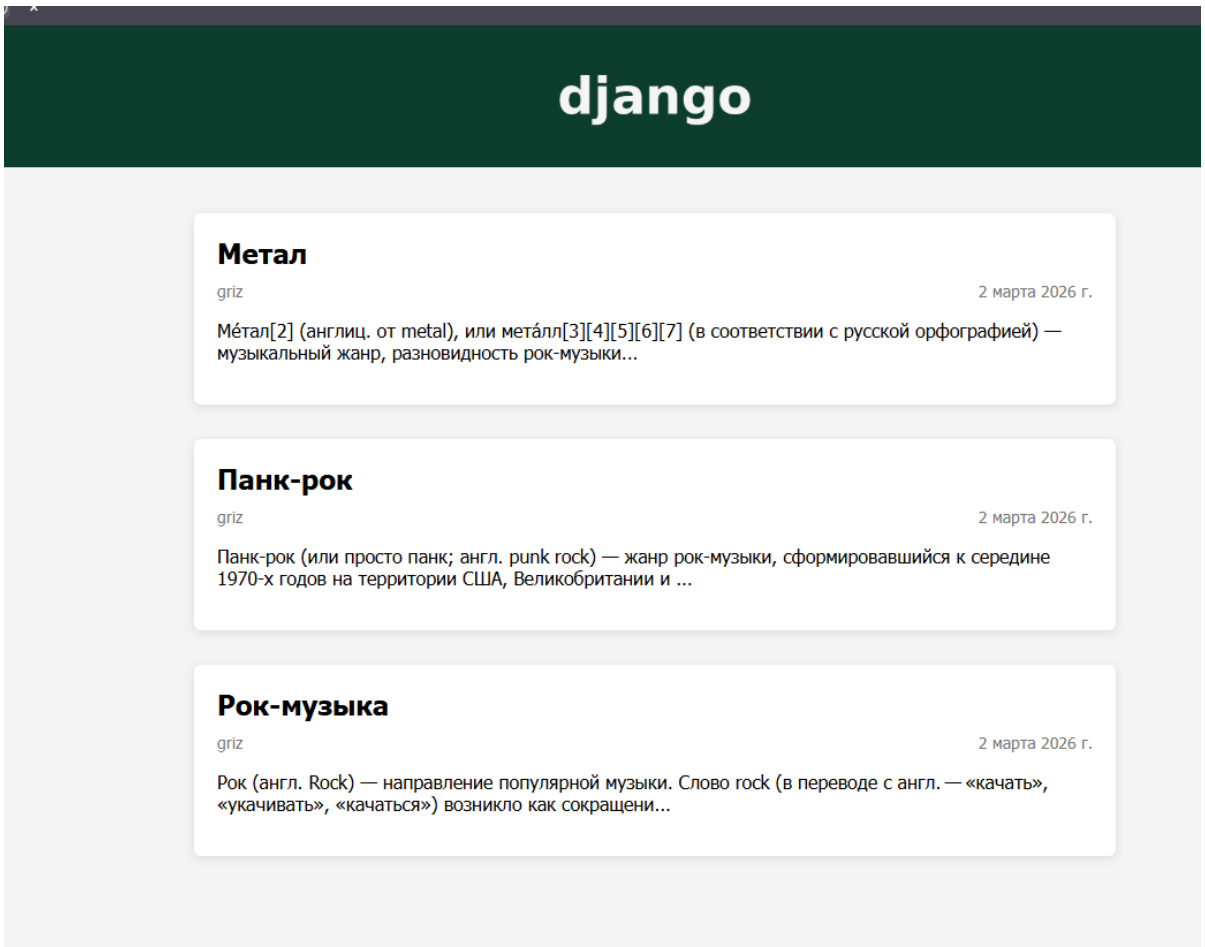
Шаблон archive.html

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
    <title>Архив всех статей</title>
    <link rel="stylesheet" href="{% static 'css/archive.css' %}">
</head>
<body>
<div class="header">
    <img src="{% static 'img/dpb-logo.png' %}" alt="logo">
</div>
<div class="archive">
    {% for post in posts %}
        <div class="one-post">
            <h2 class="post-title">{{ post.title }}</h2>
            <div class="article-info">
                <div class="article-author">{{ post.author.username }}</div>
                <div class="article-created-date">{{ post.created_date }}</div>
            </div>
            <p class="article-text">{{ post.get_excerpt }}</p>
        </div>
    {% endfor %}
</div>
</body>
</html>
```

Файл blog/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns=[path('admin/', admin.site.urls), path('', include('articles.urls'))]
```

На рисунке ниже показана страница архива, отображающая записи из базы данных.



Страница архива статей

Лабораторная работа №4

Страница определённой записи. Вёрстка в соответствии с макетом.

Цель работы

Работа направлена на создание страницы просмотра отдельной статьи блога. Для извлечения идентификатора записи из URL применяются регулярные выражения Django. Помимо этого, обе страницы (архив и статья) оформляются с помощью CSS в соответствии с предоставленными макетами.

Задачи

1. Добавить URL-маршрут с именованной группой для идентификатора статьи.

2. Написать представление get_article с обработкой исключения Article.DoesNotExist.

3. Создать HTML-шаблон для отображения отдельной статьи.

4. Стилизовать обе страницы по макетам и сделать заголовки ссылками.

Ход работы

Файл articles/views.py

```
from django.shortcuts import render, get_object_or_404, redirect
from django.http import Http404
from .models import Article

def archive(request):
    posts = Article.objects.all()
    return render(request, 'archive.html', {'posts': posts})

def get_article(request, article_id: int):
    post = get_object_or_404(Article, id=article_id)
    return render(request, 'article.html', {'post': post})
```

Шаблон article.html

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
    <title>{{ post.title }}</title>
```

```
    <link rel="stylesheet" href="{% static 'css/article.css' %}">
</head>
<body>
<div class="header">
    <a href="{% url 'archive' %}">
        <img src="{% static 'img/dpb-logo.png' %}" alt="logo">
    </a>
    <a class="all-posts-link" href="{% url 'archive' %}">Все записи</a>
</div>
<div class="content">
    <h1 class="title">{{ post.title }}</h1>
    <div class="info">
        <div class="author">{{ post.author.username }}</div>
        <div class="date">{{ post.created_date }}</div>
    </div>
    <div class="text">{{ post.text|linebreaks }}</div>
</div>
</body>
</html>
```

Шаблон archive.html

```
{% load static %}
    <!DOCTYPE html>
    <html>
    <head>
        <title>Архив всех статей</title>
        <link rel="stylesheet" href="{% static 'css/archive.css' %}">
    </head>
    <body>
    <div class="header">
    <a href="{% url 'archive' %}">
        <img src="{% static 'img/dpb-logo.png' %}" alt="logo">
    </a>
</div>

    <div class="archive">
        {% for post in posts %}
            <div class="one-post">

                    <h2 class="post-title"><a href="{% url 'get_article' article_id=post.id
%}">{{ post.title }}</a></h2>


                <div class="article-info">
                    <div class="article-author">{{ post.author.username }}</div>
                    <div class="article-created-date">{{ post.created_date }}</div>
                </div>
                <p class="article-text">{{ post.get_excerpt }}</p>

            </div>
        {% endfor %}

    </div>

    </body>
    </html>
```

CSS-файл article.css содержит стили, воспроизводящие макеты: центрированный контейнер, горизонтальное расположение автора и даты, оформление ссылок и логотипа.

Файл static/css/article.css

```
body { font-family: Tahoma, Arial, sans-serif; background: #f4f4f4; margin: 0; }
.header { background: #0c3c2c; padding: 20px; text-align: center; }
.header img { height: 60px; vertical-align: middle; }
.all-posts-link { color: #fff; margin-left: 20px; text-decoration: none; }
.content { width: 800px; margin: 40px auto; background: white; padding: 25px; border-radius:
6px; box-shadow: 0 2px 8px rgba(0,0,0,0.1); }
.info { font-size: 14px; color: #777; margin-bottom: 15px; }
.text { font-size: 16px; line-height: 1.5; }
```

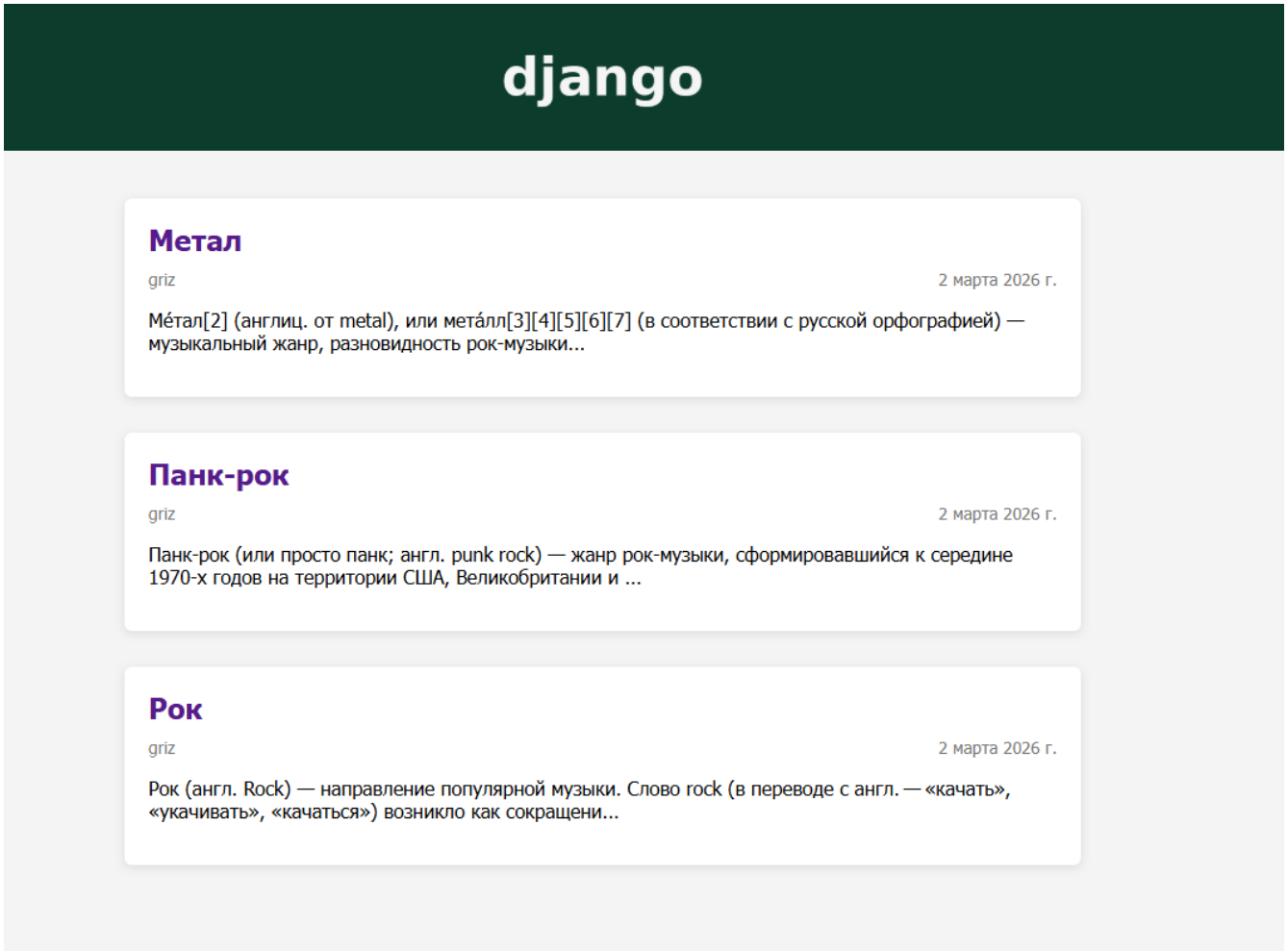
Файл static/css/archive.css

```
body { font-family: Tahoma, Arial, sans-serif; background: #f4f4f4; margin: 0; }
.header { background: #0c3c2c; padding: 20px; text-align: center; }
.header img { height: 80px; }
.auth-links a { color: #fff; margin: 0 8px; text-decoration: none; }
.archive { width: 800px; margin: 40px auto; }
.one-post { background: white; padding: 20px; margin-bottom: 30px; border-radius: 6px; box-
shadow: 0 2px 8px rgba(0,0,0,0.1); }
.post-title { margin: 0 0 10px 0; }
.post-title a { text-decoration: none; }
.article-info { font-size: 14px; color: #777; margin-bottom: 10px; }
.article-author { width: 50%; float: left; }
.article-created-date { text-align: right; }
.article-text { font-size: 16px; clear: both; }
.all-posts-link { color: #0c3c2c; text-decoration: none; display:inline-block; margin-
top:10px; }
```

Файл blog/urls.py

```
from django.contrib import admin

from django.urls import path, include

urlpatterns=[path('admin/', admin.site.urls), path('', include('articles.urls'))]
```

На рисунках представлены главная страница с кликабельными заголовками и страница отдельной статьи.



Главная страница со ссылками на статьи

Страница отдельной статьи

Лабораторная работа №5

Создание формы и представления для нового поста.

Цель работы

Лабораторная работа знакомит с механизмом обработки HTML-форм в Django. Рассматриваются отправка данных методом POST, серверная валидация введённых значений и их сохранение в базу данных. Также реализуется проверка авторизации пользователя перед созданием записи.

Задачи

1. Написать представление create_post с контролем авторизации.

2. Обработать POST-запрос с валидацией обязательных полей.

3. Разработать HTML-шаблон формы создания статьи.

4. Добавить проверку уникальности заголовка статьи.

Ход работы

Файл articles/views.py

```
from django.shortcuts import render, get_object_or_404, redirect

from django.http import Http404

from .models import Article

from .forms import ArticleForm


def archive(request):

    posts = Article.objects.all()

    return render(request, 'archive.html', {'posts': posts})


def get_article(request, article_id: int):
```

```
    post = get_object_or_404(Article, id=article_id)

    return render(request, 'article.html', {'post': post})



def create_post(request):

    if not request.user.is_authenticated:

        raise Http404

    if request.method == 'POST':

        form = ArticleForm(request.POST)

        if form.is_valid():

            post = form.save(commit=False)

            post.author = request.user

            post.save()

            return redirect('get_article', article_id=post.id)

    else:

        form = ArticleForm()

    return render(request, 'create_post.html', {'form': form})
```

Шаблон формы содержит HTML-элементы input для заголовка и textarea для текста статьи.

Шаблон create_post.html

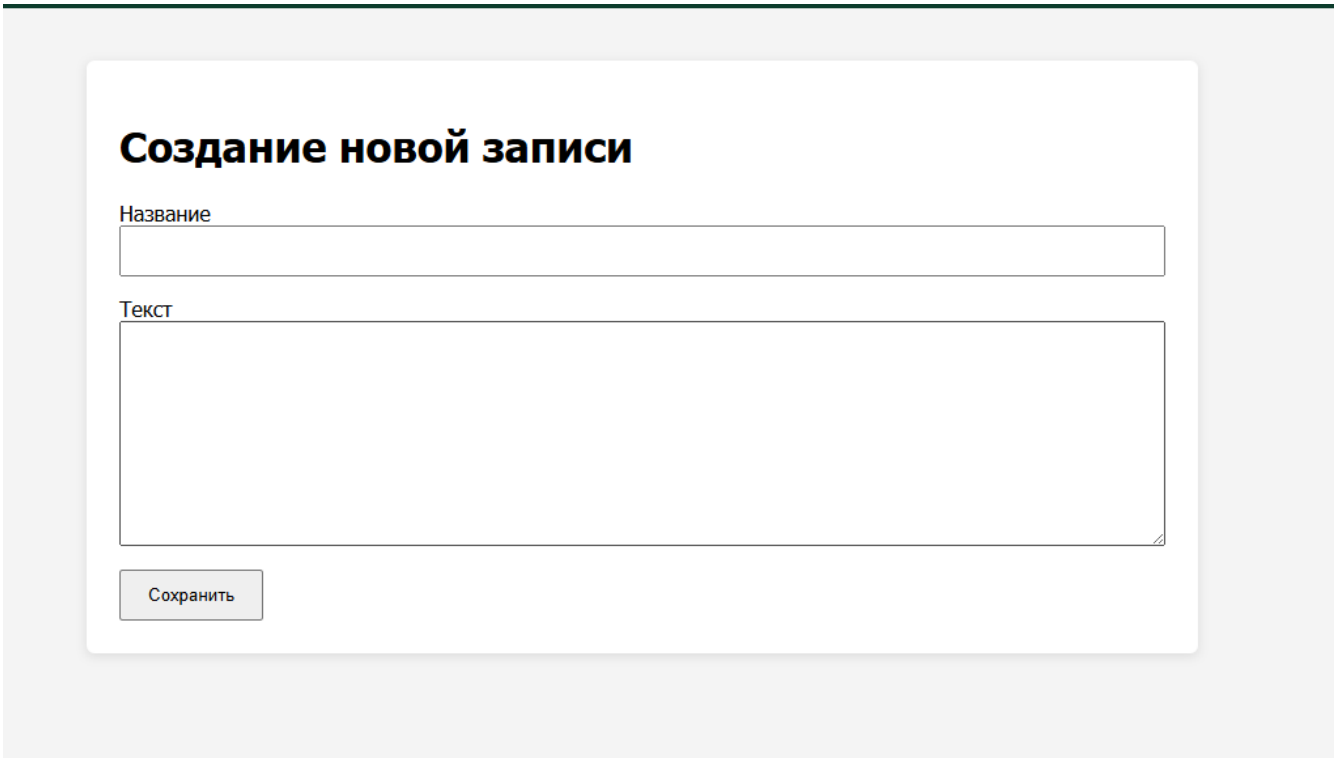
```
{% load static %}
<!DOCTYPE html>
<html>
<head>
    <title>Новая запись</title>
    <link rel="stylesheet" href="{% static 'css/create_post.css' %}">
</head>
<body>
<div class="header">
    <a href="{% url 'archive' %}">
        <img src="{% static 'img/dpb-logo.png' %}" alt="logo">
    </a>
    <a class="all-posts-link" href="{% url 'archive' %}">Все записи</a>
</div>
<div class="form-wrap">
    <h1>Создание новой записи</h1>
    <form method="post">
        {% csrf_token %}
        <label>Название</label>
        <input type="text" name="title" value="{{ form.title.value|default_if_none:'' }}">
```

```
        <label>Текст</label>
        <textarea name="text" rows="10">{{ form.text.value|default_if_none:'' }}</textarea>
        <button type="submit">Сохранить</button>
        {% if form.errors %}
        <div class="errors">
            {% for field in form %}
                {% for error in field.errors %}<p>{{ error }}</p>{% endfor %}
            {% endfor %}
        </div>
        {% endif %}
    </form>
</div>
</body>
</html>
```

Файл blog/urls.py

```
from django.urls import path
from .views import archive, get_article, create_post

urlpatterns = [
    path('', archive, name='archive'),
    path('article/<int:article_id>/', get_article, name='get_article'),
    path('article/new/', create_post, name='create_post'),
]
```

На рисунках показана форма создания новой статьи и результат — страница созданной записи.



Форма создания новой статьи

Панк-рок

griz
2 марта 2026 г.

Панк-рок (или просто панк; англ. punk rock) — жанр рок-музыки, сформировавшийся к середине 1970-х годов на территории США, Великобритании и Австралии. Истоки жанра проистекают из гаражного рока 1960-х, а также других музыкальных форм, ныне известных как протопанк. С самого начала представители жанра противопоставляли себя помпезности мейнстримного рока 1970-х: как правило, панк-группы записывали короткие, очень динамичные песни с резкими мелодиями и грубым вокалом, минимальным набором аппаратуры, а также социальными и политизированными текстами, направленными против истеблишмента. Одной из ключевых особенностей панка была DIY-идеология; многие исполнители самостоятельно продюсировали свои записи и распространяли их собственными силами или через независимые лейблы.

Впервые «панк» был использован в качестве музыкального термина в начале 1970-х. Его стали употреблять некоторые американские критики для описания гаражных групп и их поклонников. К концу 1976 года ряд исполнителей — New York Dolls, Television и Ramones из Нью-Йорка, Sex Pistols, The Clash и The Damned из Лондона — стали считаться авангардом нового музыкального движения. Годом позже жанр обрёл международную популярность и стал важным культурным явлением в Соединённом Королевстве. По большей части, панк-рок оставался в нише локальных андеграундных сцен, которые, как правило, избегали контакта с мейнстримом. В связи с популярностью жанра вокруг него возникла собственная субкультура, выраженная в протестной идеологии (юношеские бунты), отличительном стиле одежды и украшений (футболки с принтами, иногда нецензурными, кожаные куртки, различные шипованные аксессуары и одежда в БДСМ-стиле), а также антиавторитарной философии, выраженной в конфронтации с истеблишментом и этике непродажности.

К началу 1980-х в панк-роке стали преобладать более динамичные и агрессивные стили, такие как хардкор (Black Flag) и стрит-панк (Exploited). Музыканты, вдохновлённые панком или отождествлявшие себя с ним, также внесли вклад в расширение спектра его поджанров — став предтечей для появления постпанка и альтернативного рок-движения. Благодаря росту интереса к андеграундной музыке среди публики и стремлению к коммерциализации жанра со стороны мейджор-лейблов, к концу 1990-х панк-рок стал частью мейнстрима. В тот период большого успеха добились исполнители поп-панка — Green Day, The Offspring и Blink-182, — записи которых продавались многомиллионными тиражами.

Статья, созданная через форму

Лабораторная работа №6

Авторизация и регистрация пользователей.

Цель работы

Работа посвящена реализации подсистемы аутентификации. Встроенный модуль django.contrib.auth предоставляет модель пользователя, а также функции authenticate, login и метод create_user. В рамках работы создаются формы, представления и шаблоны для регистрации и входа в систему.

Задачи

1. Создать представление и шаблон для регистрации пользователей.

2. Создать представление и шаблон для страницы авторизации.

3. Реализовать валидацию полей и проверку уникальности логина.

4. Добавить навигационную ссылку на регистрацию в шапку страниц.

Ход работы

Файл articles/views.py

```
from django.shortcuts import render, get_object_or_404, redirect
from django.http import Http404
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.models import User

from .models import Article
from .forms import ArticleForm


def archive(request):
    posts = Article.objects.all()
    return render(request, "archive.html", {"posts": posts})


def get_article(request, article_id: int):
    post = get_object_or_404(Article, id=article_id)
    return render(request, "article.html", {"post": post})


def create_post(request):
    if not request.user.is_authenticated:
        raise Http404

    if request.method == "POST":
        form = ArticleForm(request.POST)
```

```
        if form.is_valid():
            post = form.save(commit=False)
            post.author = request.user
            post.save()
            return redirect("get_article", article_id=post.id)
    else:
        form = ArticleForm()

    return render(request, "create_post.html", {"form": form})


def edit_post(request, article_id: int):
    post = get_object_or_404(Article, id=article_id)

    if not request.user.is_authenticated or request.user != post.author:
        raise Http404

    if request.method == "POST":
        form = ArticleForm(request.POST, instance=post)
        if form.is_valid():
            form.save()
            return redirect("get_article", article_id=post.id)
    else:
        form = ArticleForm(instance=post)

    return render(request, "edit_post.html", {"form": form, "post": post})


def delete_post(request, article_id: int):
    post = get_object_or_404(Article, id=article_id)

    if not request.user.is_authenticated or request.user != post.author:
        raise Http404

    # GET -> показать страницу подтверждения
    # POST -> удалить
    if request.method == "POST":
        post.delete()
        return redirect("archive")

    return render(request, "delete_post.html", {"post": post})


# ---------- AUTH ----------

def register_view(request):
    if request.method == "POST":
        username = request.POST.get("username", "").strip()
        email = request.POST.get("email", "").strip()
        password = request.POST.get("password", "")

        errors = []
        if not username:
            errors.append("Введите логин")
        if User.objects.filter(username__iexact=username).exists():
            errors.append("Такой логин уже занят")
        if not password:
            errors.append("Введите пароль")

        if errors:
            return render(
                request,
                "register.html",
                {"errors": errors, "username": username, "email": email},
            )

        user = User.objects.create_user(username=username, email=email, password=password)
        login(request, user)
```

```
        return redirect("archive")

    return render(request, "register.html")


def login_view(request):
    if request.method == "POST":
        username = request.POST.get("username", "").strip()
        password = request.POST.get("password", "")

        user = authenticate(request, username=username, password=password)
        if user is None:
            return render(
                request,
                "login.html",
                {"error": "Неверный логин или пароль", "username": username},
            )

        login(request, user)
        return redirect("archive")

    return render(request, "login.html")


def logout_view(request):
    logout(request)
    return redirect("archive")
```

Шаблон registration.html

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
    <title>Регистрация</title>
    <link rel="stylesheet" href="{% static 'css/auth.css' %}">
</head>
<body>
<div class="auth-wrap">
    <h1>Регистрация</h1>
    <form method="post">
        {% csrf_token %}
        <label>Логин</label>
        <input type="text" name="username" value="{{ username|default_if_none:'' }}">
        <label>Email</label>
        <input type="email" name="email" value="{{ email|default_if_none:'' }}">
        <label>Пароль</label>
        <input type="password" name="password">
        <button type="submit">Создать аккаунт</button>
    </form>
    {% if errors %}
        {% for e in errors %}<p class="error">{{ e }}</p>{% endfor %}
    {% endif %}
    <p><a href="{% url 'login' %}">Вход</a></p>
    <p><a href="{% url 'archive' %}">На главную</a></p>
</div>
</body>
</html>
```

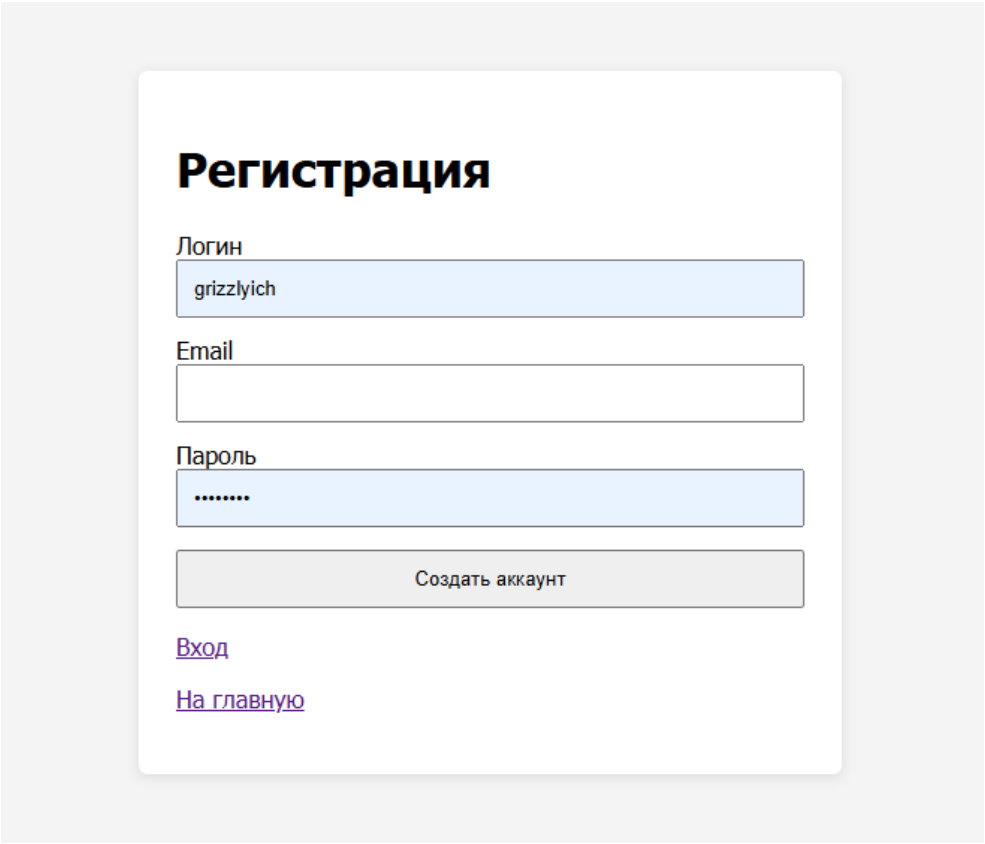
Шаблон login.html

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
    <title>Вход</title>
    <link rel="stylesheet" href="{% static 'css/auth.css' %}">
</head>
<body>
<div class="auth-wrap">
    <h1>Вход</h1>
    <form method="post">
        {% csrf_token %}
        <label>Логин</label>
        <input type="text" name="username" value="{{ username|default_if_none:'' }}">
        <label>Пароль</label>
        <input type="password" name="password">
        <button type="submit">Войти</button>
    </form>
    {% if error %}<p class="error">{{ error }}</p>{% endif %}
    <p><a href="{% url 'register' %}">Регистрация</a></p>
    <p><a href="{% url 'archive' %}">На главную</a></p>
</div>
</body>
</html>
```

Файл blog/urls.py

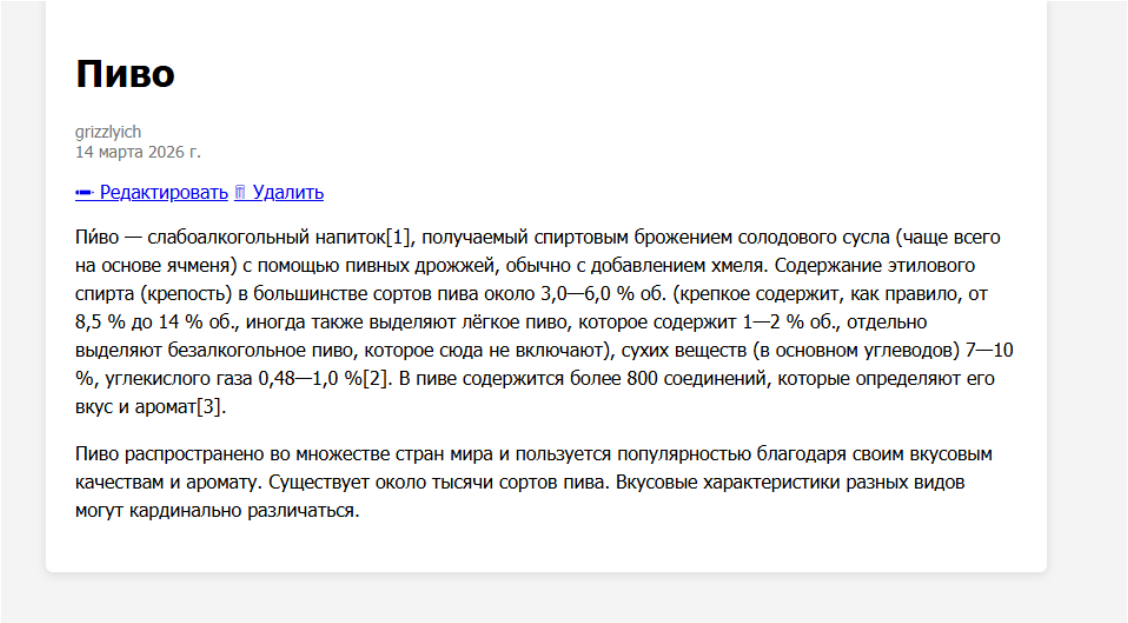
```
from django.urls import path
from .views import (
    archive,
    get_article,
    create_post,
    edit_post,
    delete_post,
    register_view,
    login_view,
    logout_view,
)

urlpatterns = [
    path("", archive, name="archive"),
    path("article/<int:article_id>/", get_article, name="get_article"),

    # создание
    path("article/new/", create_post, name="create_post"),

    # редактирование / удаление
    path("article/<int:article_id>/edit/", edit_post, name="edit_post"),
    path("article/<int:article_id>/delete/", delete_post, name="delete_post"),

    # авторизация
    path("register/", register_view, name="register"),
    path("login/", login_view, name="login"),
    path("logout/", logout_view, name="logout"),
```

На рисунке показана форма регистрации нового пользователя. Форма авторизации реализована аналогичным образом и содержит поля для ввода логина и пароля.



Форма регистрации пользователя

После регистрации и входа новый пользователь может создавать статьи от своего имени, что подтверждает корректную работу системы авторизации.



Статья, созданная зарегистрированным пользователем

Лабораторная работа №7

Основы JavaScript. Работа с DOM.

Цель работы

Лабораторная работа знакомит с языком JavaScript — его синтаксисом, типами данных и функциями. Дополнительно рассматривается взаимодействие с DOM (Document Object Model), которое позволяет изменять содержимое и поведение веб-страницы на стороне клиента без обращения к серверу.

Задачи

1. Написать JS-файл с массивом студентов и функциями вывода.

2. Добавить функцию фильтрации студентов по группе.

3. Реализовать сворачивание и разворачивание постов на главной странице.

Ход работы

Файл static/js/helloworld.js

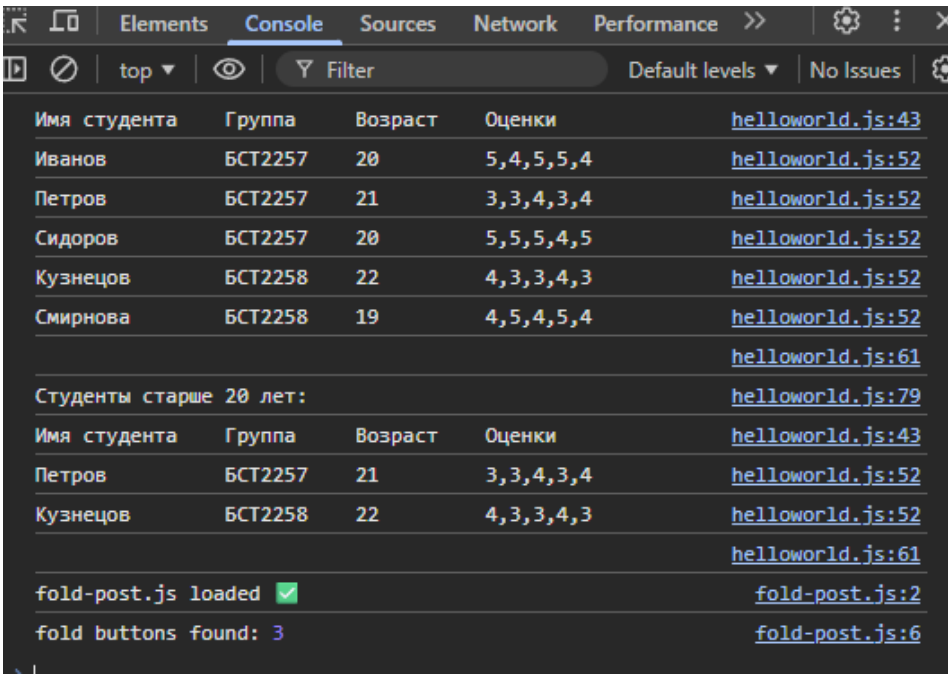
```
var groupmates = [
{
    "name": "Иванов",
    "group": "БСТ2257",
    "age": 20,
    "marks": [5,4,5,5,4]
},
{
    "name": "Петров",
    "group": "БСТ2257",
    "age": 21,
    "marks": [3,3,4,3,4]
},
{
    "name": "Сидоров",
    "group": "БСТ2257",
    "age": 20,
    "marks": [5,5,5,4,5]
},
{
    "name": "Кузнецов",
    "group": "БСТ2258",
    "age": 22,
    "marks": [4,3,3,4,3]
},
```

```
{
    "name": "Смирнова",
    "group": "БСТ2258",
    "age": 19,
    "marks": [4,5,4,5,4]
}
];

var rpad = function(str, length){
    str = str.toString();
    while (str.length < length)
        str = str + ' ';
    return str;
};

var printStudents = function(students){

    console.log(
        rpad("Имя студента",15),
        rpad("Группа",10),
        rpad("Возраст",10),
        rpad("Оценки",20)
    );

    for (var i = 0; i < students.length; i++){

        console.log(
            rpad(students[i]['name'],15),
            rpad(students[i]['group'],10),
            rpad(students[i]['age'],10),
            rpad(students[i]['marks'],20)
        );

    }

    console.log('\n');
};

var filterByAge = function(students, minAge){

    var result = [];

    for (var i = 0; i < students.length; i++){
        if (students[i]['age'] >= minAge){
            result.push(students[i]);
        }
    }

    return result;
};

printStudents(groupmates);

console.log("Студенты старше 20 лет:");

printStudents(filterByAge(groupmates,21));
```

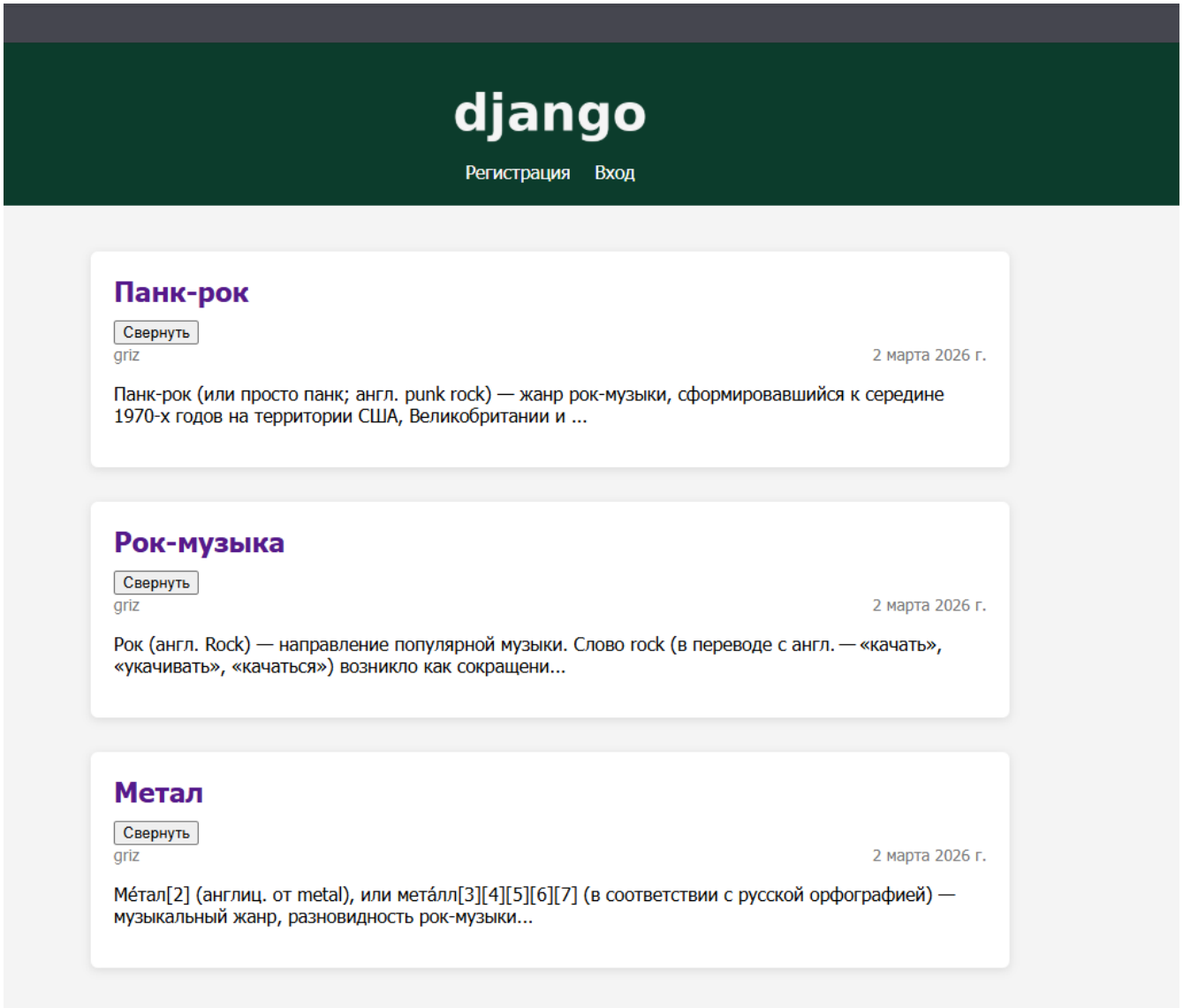
Файл static/js/fold-post.js

```
window.__FOLD_SCRIPT_LOADED__ = true;
console.log("fold-post.js loaded ✅");

document.addEventListener("DOMContentLoaded", () => {
  const buttons = document.querySelectorAll("button.fold-button");
  console.log("fold buttons found:", buttons.length);

  buttons.forEach((btn) => {
    btn.addEventListener("click", () => {
      const post = btn.closest(".one-post");
      if (!post) return;

      post.classList.toggle("folded");

      btn.textContent = post.classList.contains("folded")
        ? "Развернуть"
        : "Свернуть";
    });
  });
});
```

На рисунках показаны консоль разработчика с результатом выполнения JS-скрипта и главная страница с кнопками сворачивания постов.



Консоль разработчика с выводом скрипта helloworld.js



Главная страница с кнопками сворачивания

Лабораторная работа №8

Библиотека jQuery. Подсветка постов и эффект для логотипа.

Цель работы

Работа направлена на освоение библиотеки jQuery, упрощающей манипуляции с DOM, обработку событий и анимации. В рамках задания подключается jQuery через CDN и реализуются визуальные эффекты: подсветка поста при наведении и анимация логотипа.

Задачи

1. Подключить jQuery к проекту через CDN.

2. Реализовать подсветку фона поста при наведении курсора.

3. Создать анимационный эффект для изображения логотипа.

Ход работы

Файл static/js/jquery-effects.js

```
$(function () {
    // --- Подсветка поста ---
    $('.one-post').hover(
        function () {
            $(this).css('background-color', '#ebf5e7');
            $(this).css('border', '1px solid #0c3c2c');
        },
        function () {
            $(this).css('background-color', '#ffffff');
            $(this).css('border', '1px solid #dddddd');
        }
    );

    // --- Логотип: увеличение по ширине на 20px, высота пропорционально ---
    const $logo = $('.header img');
    if ($logo.length) {
        const originalWidth = $logo.width();
        const originalHeight = $logo.height();
        const ratio = originalHeight / originalWidth;

        $logo.hover(
            function () {
                $(this).stop(true, true).animate({
                    width: originalWidth + 20,
                    height: originalHeight + (20 * ratio)
                }, 150);
            },
```

```
            function () {
                $(this).stop(true, true).animate({
                    width: originalWidth,
                    height: originalHeight
                }, 150);
            }
        );
    }
});
```

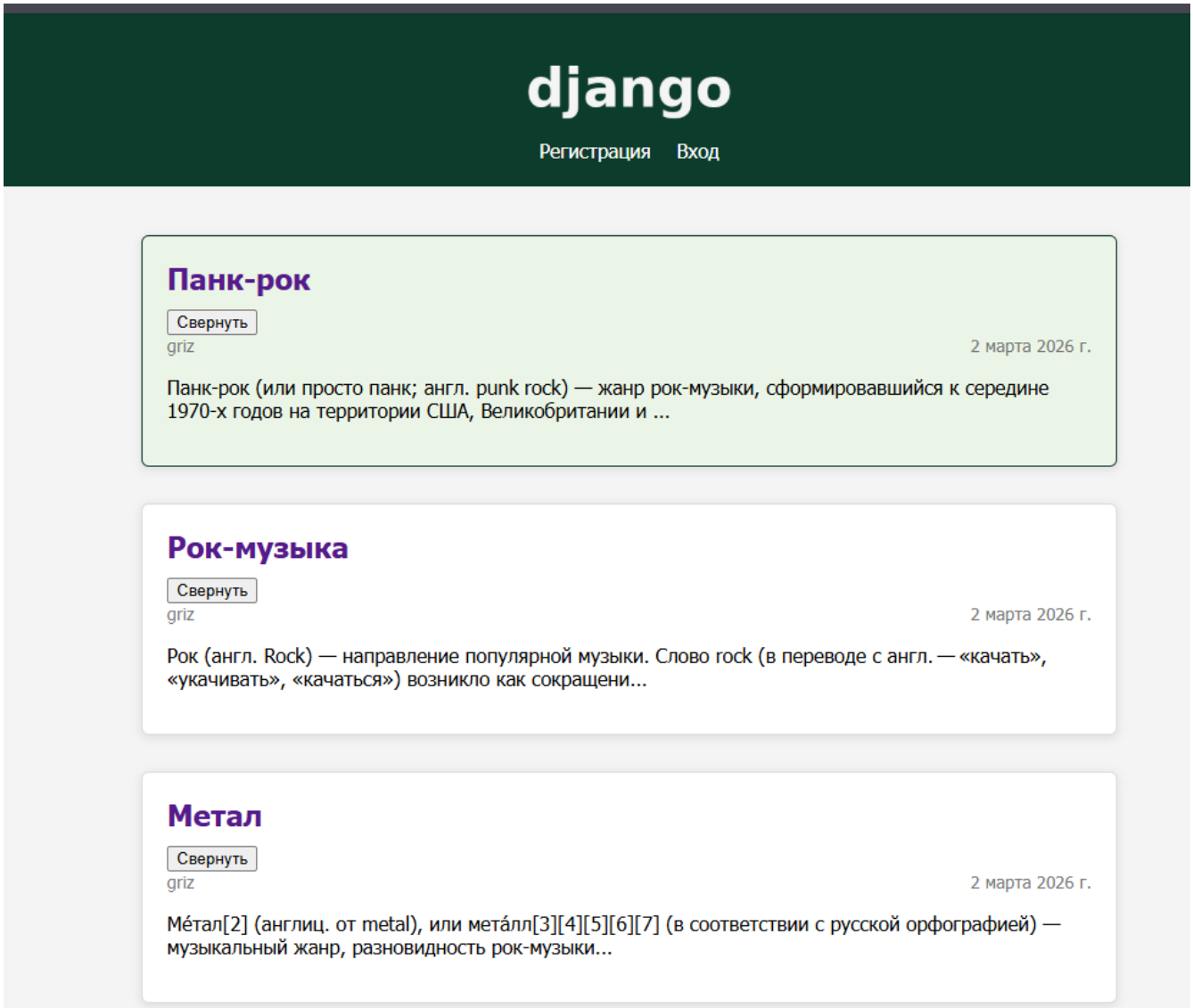
Шаблон archive.html

```
{% load static %}

<!DOCTYPE html>

<html>

<head>

    <title>Архив всех статей</title>

    <link rel="stylesheet" href="{% static 'css/archive.css' %}">

</head>

<body>



<div class="header">

    <a href="{% url 'archive' %}">

        <img src="{% static 'img/dpb-logo.png' %}" alt="logo">

    </a>



    <div class="auth-links">

        {% if request.user.is_authenticated %}

            <span class="hello">Привет, {{ request.user.username }}</span>

            <a href="{% url 'logout' %}">Выход</a>

        {% else %}

            <a href="{% url 'register' %}">Регистрация</a>

            <a href="{% url 'login' %}">Вход</a>

        {% endif %}
```

```
            </div>
        </div>


        <div class="archive">
            {% for post in posts %}
                <div class="one-post">
                    <div class="post-managment">
                        <h2 class="post-title">
                            <a href="{% url 'get_article' article_id=post.id %}">{{ post.title
}}</a>
                        </h2>
                        <button type="button" class="fold-button">Свернуть</button>
                    </div>


                    <div class="article-info">
                        <div class="article-author">{{ post.author.username }}</div>
                        <div class="article-created-date">{{ post.created_date }}</div>
                    </div>


                    <p class="article-text">{{ post.get_excerpt }}</p>


                    {% if request.user.is_authenticated and request.user == post.author %}
                        <div class="post-actions">
                            <a class="btn" href="{% url 'edit_post' article_id=post.id %}">✏
Редактировать</a>
                            <a class="btn danger" href="{% url 'delete_post' article_id=post.id
%}">🗑 Удалить</a>
                        </div>
                    {% endif %}
```

```
            <div class="shadow"></div>
        </div>
    {% empty %}
        <p>Пока нет записей.</p>
    {% endfor %}


    {% if request.user.is_authenticated %}
        <a class="all-posts-link" href="{% url 'create_post' %}">Новая запись</a>
    {% endif %}
</div>


<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<script src="{% static 'js/highlight-post.js' %}"></script>
<script src="{% static 'js/fold-post.js' %}"></script>
</body>
</html>
```

Результат работы — при наведении курсора фон поста изменяется:



Подсветка поста при наведении курсора

Лабораторная работа №9

jQuery Parallax. Эффект параллакса для главной страницы.

Цель работы

Заключительная лабораторная работа расширяет навыки работы с jQuery. Реализуется эффект параллакса — визуальный приём, при котором фоновое изображение смещается медленнее основного содержимого, создавая ощущение глубины при прокрутке страницы.

Задачи

1. Реализовать эффект параллакса для фонового изображения главной страницы.

2. Обновить CSS-стили для поддержки параллакс-эффекта.

Ход работы

Файл static/js/parallax.js

```
$(function () {
    const $icons = $('.icons-for-parallax img');
    const $logo = $('.logo');
    let ticking = false;

    function updateParallax() {
        const scrollTop = $(window).scrollTop();

        $icons.each(function (index) {
            const speed = [0.12, 0.20, 0.28][index] || 0.15;
            const offsetY = scrollTop * speed;
            $(this).css('transform', 'translate3d(0,' + offsetY + 'px,0)');
        });

        const logoOffset = scrollTop * 0.06;
        $logo.css('transform', 'translate3d(0,' + logoOffset + 'px,0)');
        ticking = false;
    }

    $(window).on('scroll', function () {
        if (!ticking) {
            window.requestAnimationFrame(updateParallax);
            ticking = true;
        }
    });
```

```
    updateParallax();
});
```

Файл static/css/article.css

```
body { font-family: Tahoma, Arial, sans-serif; background: #f4f4f4; margin: 0; }

.header { background: #0c3c2c; padding: 20px; text-align: center; }
.header img { height: 60px; vertical-align: middle; }
.all-posts-link { color: #fff; margin-left: 20px; text-decoration: none; }

.content {
    width: 900px;
    margin: 40px auto;
    background: white;
    padding: 25px;
    border-radius: 6px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}

.title { margin-top: 0; }

.info { font-size: 14px; color: #777; margin-bottom: 15px; display:flex; justify-content:
space-between; }
.text { font-size: 16px; line-height: 1.5; }

.post-actions { margin: 15px 0 20px 0; }
.inline-form { display: inline; }

.btn {
    display: inline-block;
    padding: 10px 16px;
    background: #0c3c2c;
    color: white;
    text-decoration: none;
    border-radius: 6px;
    margin-right: 10px;
    border: none;
    cursor: pointer;
    font-size: 14px;
}

.btn.danger { background: #b00020; }
.btn.light { background: #666; }
```

Шаблон archive.html адаптирован для параллакса — добавлен контейнер с фоновым изображением и подключён скрипт parallax.js.

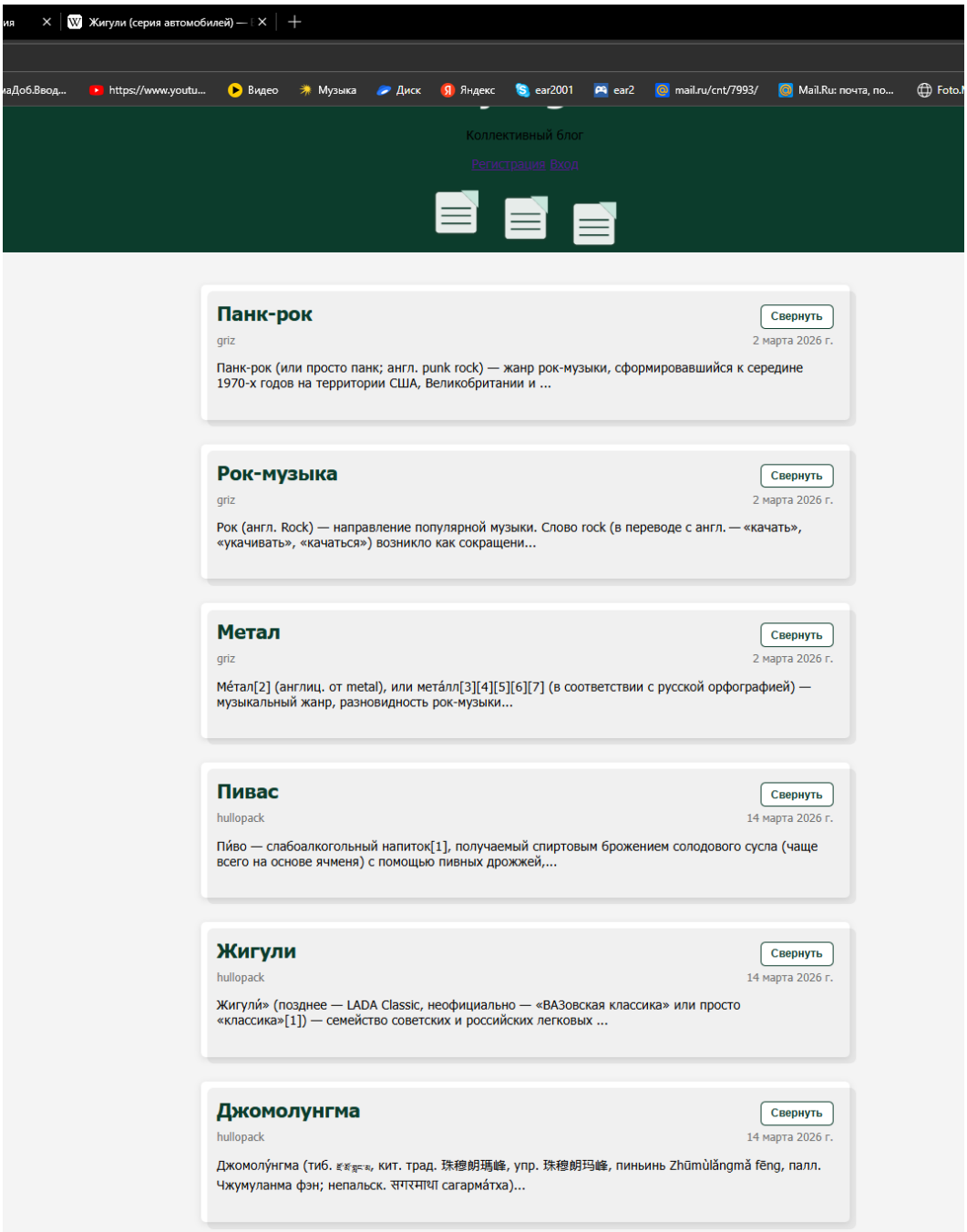
Шаблон archive.html

```
{% load static %}
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Архив всех статей</title>
    <link rel="stylesheet" href="{% static 'css/archive.css' %}">
</head>
<body>
```

```
<div class="header">
    <div class="header-inner">
        <a href="{% url 'archive' %}" class="logo-link">
            <img class="logo" src="{% static 'img/dpb-logo.png' %}" alt="logo">
        </a>

        <div class="header-text-area">
            <p class="service-title">Коллективный блог</p>
            {% if request.user.is_authenticated %}
                <span class="hello">Привет, {{ request.user.username }}</span>
                <a href="{% url 'logout' %}" class="register-link">Выход</a>
            {% else %}
                <a href="{% url 'register' %}" class="register-link">Регистрация</a>
                <a href="{% url 'login' %}" class="register-link">Вход</a>
            {% endif %}
        </div>

        <div class="icons-for-parallax" aria-hidden="true">
            <img class="icon-for-parallax-first" src="{% static 'img/article-icon-for-
parallax.png' %}" alt="">
            <img class="icon-for-parallax-second" src="{% static 'img/article-icon-for-
parallax.png' %}" alt="">
            <img class="icon-for-parallax-third" src="{% static 'img/article-icon-for-
parallax.png' %}" alt="">
        </div>
    </div>
</div>

<div class="page-content">
    <div class="archive">
        {% for post in posts %}
            <div class="one-post">
                <div class="shadow"></div>

                <div class="post-managment">
                    <h2 class="post-title">
                        <a href="{% url 'get_article' article_id=post.id %}">{{ post.title
}}</a>
                    </h2>
                    <button type="button" class="fold-button">Свернуть</button>
                </div>

                <div class="article-info">
                    <div class="article-author">{{ post.author.username }}</div>
                    <div class="article-created-date">{{ post.created_date }}</div>
                </div>

                <p class="article-text">{{ post.get_excerpt }}</p>

                {% if request.user.is_authenticated and request.user == post.author %}
                    <div class="post-actions">
                        <a class="btn" href="{% url 'edit_post' article_id=post.id %}">✏
Редактировать</a>
                        <a class="btn danger" href="{% url 'delete_post' article_id=post.id
%}">🗑 Удалить</a>
                    </div>
                {% endif %}
            </div>
        {% empty %}
            <p class="empty">Пока нет записей.</p>
        {% endfor %}

        {% if request.user.is_authenticated %}
            <a class="all-posts-link" href="{% url 'create_post' %}">Новая запись</a>
        {% endif %}
    </div>
</div>
```

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<script src="{% static 'js/helloworld.js' %}"></script>
<script src="{% static 'js/fold-post.js' %}"></script>
<script src="{% static 'js/highlight-post.js' %}"></script>
<script src="{% static 'js/parallax.js' %}"></script>
</body>
</html>
```

Итоговый вид главной страницы с эффектом параллакса:



Главная страница с эффектом параллакса